

*Note: Claude was used in the creation of these tools. If you're not comfortable with LLM generated code, the previous MD5 tools are safe for use. As to the rest of these tools - this was my first dive into AI coding, and I admit to getting a little crazy with the cheez wiz.

idTech4 Blender Tools

Import, edit and export idTech4 game assets in Blender — models, animations, materials, normal maps, camera files, .map files.

This is a collection of Blender add-ons designed for working with idTech4 assets (**Doom 3**, **Doom 3 BFG**, **Quake 4**, **Prey** and **The Dark Mod**) The tools are designed with the idTech4 engine in mind - for example .lwo and .ase import and export have validation checking and feature implementations based on engine requirements. Each addon can install and run on its own; together they can open a .map file with textured geometry, static models, and animated characters carrying their attachments.

Format behaviours were verified against the Doom 3 GPL sources.

Contents

- [Features](#)
 - [Requirements](#)
 - [Installation](#)
 - [First-time setup](#)
 - [The add-ons](#)
 - [Using them together](#)
 - [Supported formats](#)
 - [Documentation](#)
 - [Project layout](#)
 - [Development](#)
 - [License](#)
-

Features

Models

- Import and export **.ase** and **.lwo** static meshes
- Import and export **.md5mesh** skeletal meshes and **.md5anim** animations, in both Version 10 and Version 12
- Import and export **.md5camera** camera paths, with cuts as timeline markers
- Read the Doom 3 BFG compiled binary files: **.bmd5mesh**, **.bmd5anim**, **.base**, **.blwo**, **.bimage**
- Auto generate blender materials and shader nodes from model material definitions
- Model validation - export check that reports what the engine will reject, with fix actions
- An animation retargeter for correcting animations after bone orientation edits

- A bone-collection manager controlling exactly which bones reach the file

Materials and textures

- Builds fully textured Blender materials/shaders from `.mtr` declarations
- Tested against Doom 3, BFG, Quake 4, Prey and Dark Mod assets
- Four fidelity modes when creating materials to improve performance in complex scenes.
- Support for dynamic materials via blender driver use. Scale, rotation, table lookups, renderparms and time are supported.
- Full image-program support: `heightmap`, `addnormals`, `smoothnormals`, `makeintensity`, `invertcolor`, `scale` and the rest, nested as the engine does
- Finds, fades and hides editor-texture materials — trigger volumes, clip brushes, visportals
- Decodes Doom 3 BFG `.bimage` files, including DXT5-normal, YCoCg and green-alpha packings

Levels

- Import `.map` files — brushes, patches and entities as Blender meshes with materials and UVs
- Read id Tech 4 `brushDef3`, `patchDef2`, and `patchDef3` Bezier patches
- Place static and skeletal models
- Place characters' heads and attachments — the models `def_head` and `def_attach*` declare — bone-parented to the body's skeleton
- Apply `.skin` material swaps per placed entity
- Export worldspawn brush geometry back to `.map` as `brushDef3`, Version 2 or Version 3
- Validate geometry against what a `brushDef3` can actually hold

Normal Map Baking

- A re-implementation of the engine's `renderbump` normal-map bake tool

Requirements

Blender	4.5 or newer recommended. Tested on Blender 4.x thru 5.x.
Dependencies	None. Standard library and <code>bpy</code> only — no <code>pip install</code> step.
Game assets	Extracted to a normal directory tree. See First-time setup .

Individual add-ons declare lower minimums (RenderBump works from 3.2, MD5 Tools from 4.0), but install the set at 4.5+.

Installation

Install only the add-ons you need — each works alone.

1. Download the `.py` files from this repository.
2. In Blender, open **Edit** → **Preferences** → **Add-ons**.
3. Click **Install...**, select a `.py` file, and confirm.
4. Tick the checkbox beside it to enable it.

5. Repeat for each add-on you want.

File	Enable the entry named
<code>idTech4_MD5_Tools.py</code>	idTech4 MD5 tools - .md5mesh,md5anim, and md5camera Import/Export
<code>idTech4_ase_lwo_io.py</code>	idTech4 .ase / .lwo Importer/Exporter
<code>idTech4_map_io.py</code>	idTech4 .map Importer
<code>idTech4_material_import.py</code>	idTech4 Materials
<code>idTech4_renderbump.py</code>	idTech4 RenderBump
<code>idTech4_bimage.py</code>	idTech4 Binary Image (.bimage)

[!TIP] Add-on availability is re-checked continuously, so enabling one takes effect immediately — no Blender restart needed.

First-time setup

[!IMPORTANT] **These add-ons work on extracted game assets.** id Tech 4 games ship their content inside archives — `.pk4` for Doom 3, Quake 4, Prey and The Dark Mod, `.resources` for Doom 3 BFG. Nothing here reads inside an archive.

A `.pk4` is an ordinary zip archive that any zip tool will open. A `.resources` file is not, and needs a Doom 3 BFG extraction tool.

Extract every archive into one folder, **preserving the paths inside them**, so the result is the real id Tech 4 directory structure:

```
<game>/
  base/           ← this is your Base Directory
    def/          entityDef and model declarations
    dds/          precompressed textures
    guis/
    maps/         .map source files
    materials/    .mtr material declarations
    models/       .md5mesh, .md5anim, .ase, .lwo
    skins/        .skin declarations
    textures/     .tga / .png / .jpg source textures
    generated/    BFG only: .bimage, .bmd5mesh, .base, .blwo caches
```

Then point the add-ons at it:

1. In the 3D Viewport press N to open the sidebar.
2. Go to the **idTech4** tab → **Sources** panel.
3. Set **Base Directory** to the `base` folder. This is the only mandatory setting.

4. Optionally set **Mod Base Directory** to a mod or expansion folder — it is searched first, with Base Directory still searched behind it, per item. This is the engine's own `fs_game` behaviour.
5. Leave **Materials Source** blank unless your `.mtr` files are somewhere other than `<base>/materials`.

► Details that catch people out

If you skip this and then tick a material or model option, the import stops and offers to set the paths up for you — pick them now, derive them from the file being imported, or skip materials for that run.

The add-ons

 **MD5 Tools** — skeletal models, animation and cameras

`idTech4_MD5_Tools.py` · **File** → **Import/Export** → **idTech4 MD5 ...** · sidebar tab **MD5**

Full import and export of `.md5mesh`, `.md5anim` and `.md5camera`, in both the original Version 10 format and the extended Version 12 format with normals, tangents and vertex colours. Reads the BFG binary formats `.bmd5mesh` and `.bmd5anim`, and both can be mixed freely with text files in one selection.

- **Import mesh** — creates a collection holding the armature and every mesh, with optional material generation
- **Import animation** — multi-select any number of `.md5anim` files; each becomes one Action
- **Export** mesh, animation, or both in a single combined dialog
- **Cameras** — each `.md5camera` becomes its own camera object with keyframed location, rotation and FOV; cuts become timeline markers
- **Bone Collection Manager** (*Properties* → *Data*) — only bones in `MD5_export_bone_collection` are exported, so a control rig can sit on top of the deformation skeleton without corrupting the file
- **Animation Retargeter** (*sidebar*) — snapshot the rest pose, realign bone orientations, then recompute every keyframe in every action so the visual pose is preserved
- Animation compression with a configurable delta threshold, and per-frame bounding-box scaling

 **ASE / LWO Tools** — static meshes

`idTech4_ase_lwo_io.py` · **File** → **Import/Export** → **idTech4 ASE / LWO** · sidebar tab **idTech4**

Import and export both static mesh formats the engine reads, plus the BFG binary formats `.base` and `.blwo`.

- **Four shading modes** — *File Shading* (what the file intends), *Engine Shading* (what id Tech 4 will actually render), plus Smooth and Flat
- **Build Engine Render Mesh** — reproduces the engine's own weld and split behaviour so you can see exactly where your model will be split
- **Lossless .lwo round trip** — a Blender vertex index *is* the `.lwo` point index, so a file can be imported, edited and written back unchanged
- **Export Check panel** — validates against the engine's real acceptance rules and groups findings into what needs a decision from you, what *Prepare for Export* can fix, and what the exporter handles anyway
- **Prepare for Export** — triangulates and removes zero-area faces, optionally working on a copy so your authoring cage survives
- Multi-layer `.lwo` output, LWO smoothing groups, full vertex-map support, and exact per-corner normals in `.ase`

Map Tools — levels

`idTech4_map_io.py` · **File** → **Import/Export** → **idTech4 .map** · sidebar tab **idTech4 Map**

- Imports `brushDef3`, Quake 3 and Valve 220 face formats and both patch types
- One collection per entity classname; every spawnarg preserved as a `map_*` custom property
- **Worldspawn grouping** — combine thousands of brushes into a single object, or keep them separate
- **Model placement** — static and skeletal, with three animation modes trading fidelity against viewport speed
- **Heads and attachments** — a character is several models bound to joints of the body's skeleton, and they follow it through an animation
- **Export** worldspawn brushes back to `.map`, with texture matrices *recovered* rather than approximated — a face whose UVs are not one flat projection is refused rather than silently reshuffled
- **Validate Collection panel** — checks meshes for the four things a `brushDef3` must be (closed, flat-faced, convex, one material and one flat UV projection per plane) and selects any offending elements for easy identification in the viewport

Materials — `.mtr` material generation

`idTech4_material_import.py` · sidebar tab **idTech4 Mtr**

Both a panel you use directly and what every other addon calls for materials.

Fidelity mode	What it builds
Maximum	The renderer's own draw sequence — one Principled BSDF per interaction pass, specular, cube maps, ambient stages through their blend equations
Good	1.14× faster. Interaction passes collapsed onto a Diffuse BSDF; keeps normals, heightmaps, cube maps and the roughness estimate
Basic	1.30× faster. No specular texture loaded at all; only ambient stages carrying alpha
Simple	1.50× faster, a third of the texture memory. Diffuse texture on a lit Diffuse BSDF
Parameter policy	Effect
Baked	Every expression folded once. No drivers, nothing re-runs per frame. Use for whole-map imports.
Dynamic	Drivers for <code>time</code> , live re-folding for other engine parameters. Materials animate and respond to the parm sliders.
Skip	Refuse every parameter and conditional. Cheapest and least faithful.

Also: an **Editor Textures** section that finds every material that is a level-editor placeholder and lets you fade or hide them; shader parm sliders materials respond to live; a material table inventory; a node-graph topology inventory; and a full build report for every material that fell back to a placeholder and why.

RenderBump — normal map baking

idTech4_renderbump.py · sidebar tab **idTech4 Renderbump**

A re-implementation of the engine's **renderbump** console command, including the exact model-loading path it takes to get there.

This is a **re-implementation, not an improvement**. Every quirk is kept deliberately: 100-step ray march, farthest-hit-wins trace, truncating float-to-byte conversions. Against a reference map built by the original tool: **99.25% of texels bit-identical, 99.996% within one 8-bit step**. The residue is the SSE **rsqrtps** approximation, which is a property of the CPU and not reproducible in software.

Reads its low and high poly from scene objects or from model files on disk.

Binary Image — BFG texture caches

idTech4_bimage.py · **File** → **Import** → **idTech4 Binary Image**

Decodes Doom 3 BFG's compiled **.bimage** texture cache into an ordinary Blender image. Handles RGBA8, XRGB8, ALPHA, LUM8, L8A8, INT8, RGB565, DXT1, DXT5 and the PS4 build's BC5 and BC7, plus the DXT5-normal, YCoCg and green-alpha channel packings. Colorspace is set automatically from the file's own colour format, so a decoded normal map is immediately usable.

Using them together

Every add-on can work independently, but when several are installed, they cooperate:

Map Tools → ASE / LWO Tools	places the static models a map references
→ MD5 Tools	places skeletal models, animation, heads, attachments
→ Materials	builds materials for every face
MD5 Tools → Materials	builds materials for an imported model
ASE / LWO → Materials	builds materials for an imported model

[!NOTE] When a companion add-on is missing, the option that needs it is **greyed out with a message naming it**

Small shared helpers are duplicated verbatim in each addon rather than imported, so the add-ons install independently — but this means **the addons must be kept at matching versions**.

► **Sidebar tabs**

► **Recommended settings for a whole-level import**

Supported formats

Format	Extensions	Import	Export
Level source	.map	✓	✓ worldspawn brushes
Skeletal mesh	.md5mesh .bmd5mesh	✓	✓ text only

Format	Extensions	Import	Export
Skeletal animation	.md5anim .bmd5anim	✓	✓ text only
Camera path	.md5camera	✓	✓
Static mesh	.ase .base	✓	✓ text only
Static mesh	.lwo .blwo	✓	✓ text only
Materials	.mtr	✓	—
Entity / model decls	.def	✓	—
Skins	.skin	✓	—
Texture cache	.bimage	✓	—

Games: Doom 3 · Doom 3 BFG · Quake 4 · Prey · The Dark Mod

Documentation

Each add-on has full reference documentation:

Document	Covers
MD5Tools_Documentation.docx	MD5 Tools
ASE_LWO_Tools_Documentation.docx	ASE / LWO Tools
Map_Tools_Documentation.docx	Map Tools
Materials_Documentation.docx	Materials
RenderBump_Documentation.docx	RenderBump
BImage_Documentation.docx	Binary Image

Project layout

.

|— idTech4_MD5_Tools.py

|— idTech4_ase_lwo_io.py

|— idTech4_map_io.py

|— idTech4_material_import.py

|— idTech4_renderbump.py

|— idTech4_bimage.py

|— *.docx

the six add-ons, each installable on its own

per-add-on reference documentation

License

`idTech4_renderbump.py` is [GPL-3.0-or-later](#), as declared in its SPDX header — it is a direct re-implementation of code from the Doom 3 GPL release.

Credits

Built by **Samson**, with Claude.

Engine behaviour throughout is derived from the **Doom 3 GPL source release** by id Software, and from **The Dark Mod**'s renderer for the parts where it diverges.